

Midterm/ Final Exam

Course Title	Data structures and Algorithms		Course Code	ELE5252	
Semester	Fall		Academic Year	2022/2023	
Exam Date	04/12/2022	Day	Sunday	Time	11:30-12:20

Student Name		Student ID No.		Section	
--------------	--	----------------	--	---------	--

GENERAL INSTRUCTIONS

- Students are not allowed to leave the exam room during the first 30 minutes of the exam.
- Late students are not permitted to complete their exams after the first 30 minutes unless authorized by the proctor (no students left the exam session), knowing that no additional time will be granted.
- Students must use a blue or black pen. Pencils are not allowed.
- Consumption of any food or beverage is prohibited except in medical cases.
- All electronic devices (e.g., cell phones) are prohibited.
- All cell phones must be turned off and placed on the invigilator's desk in the exam room.
- All notes and reading materials must be placed on the invigilator's desk in the exam room.
- Students are not permitted to speak to or ask questions of their classmates. If a student has a question, he or she must raise his or her hand to be assisted by the proctor.
- Write the full information required on this page before commencing your exam.
- Students are not allowed to leave the room once the exam commences for any reason unless it is absolutely necessary.
- Reported student cheating cases, misbehavior, or other student misconduct on examination sessions will be reported to the Investigations Committee as per internal bylaws.

I have understood and abide with the exam policy. **Student's signature:** _____

Question No.	Question Type	CILOs assessed	Marks Weight	Scored Marks	Marker signature	Student Score
1	Multiple Choice	CILO 1, 2, 9, 11, 13	10(33.3%)			
2	Essay	CILO 1, 2, 4, 9, 10, 13	6(20%)			
3	Essay	CILO 1→3, 9, 11, 13	5(16.7%)			
4	Essay	CILO 3, 4, 8, 13, 14	9(30%)			
Total			30(100%)			

Moderation Review

Comments					
Marks Reviewed by				Signature & Date	

Problem # 1: Multiple choice question. [10 x 1pt]

- 1) The value of REAR is increased by 1 when
 - A. An element is deleted in a queue
 - B. An element is traversed in a queue
 - C. An element is added in a queue**
 - D. An element is merged in a queue

- 2) The deletion operation in stack is called
 - A. insert
 - B. push
 - C. pop**
 - D. enqueue

- 3) Link fields holds pointers to the element in the linked representation of stack.
 - A. Neighboring**
 - B. Last
 - C. First
 - D. Middle

- 4) Which of the following is **TRUE** about destructor?
 - A. Its name is the class name preceded by a ~ (tilde)
 - B. It is a special member function that works just opposite to constructor
 - C. Destructors are used to destroy (or delete) the object
 - D. All of the above**

- 5) Which of the following is **FALSE** about Circular Queue?
 - A. It helps in refilling all the free spaces by resetting REAR to zero when the REAR reaches the end of the queue
 - B. It follows the Last in First out LIFO (LIFO) principle**
 - C. It connects the last position of the queue to the first position of the queue
 - D. None of the above

- 6) A destructor is automatically called when
 - A. The program finished execution.
 - B. Calling the delete operator.
 - C. A scope (the { } parenthesis) containing local variable ends.
 - D. All of the above**

- 7) Given the class definition in the table below, which data member you cannot call by an object and the dot operator from outside the class

```
class MyClass{
    public: int x,y;
    private:int a;
};
```

A. x

B. y

C. a

D. None of the above

- 8) Class and function templates allow the programmer to write one code for all data types and help in generic programming.

A) True

B) False

- 9) Which of the following is **TRUE** about linear Queue?

A. It is also called Last-In-Last-Out list (LILO)

B. It becomes full when Front = size -1

C. It becomes empty only when Rear = -1 and Front = -1.

D. None of the above

- 10) In doubly linked list, items can be navigated forward and backward.

A. True

B. False

*** Write the correct answers of the problem # 1 in the table below:

<u>Q1</u>	<u>Q2</u>	<u>Q3</u>	<u>Q4</u>	<u>Q5</u>	<u>Q6</u>	<u>Q7</u>	<u>Q8</u>	<u>Q9</u>	<u>Q10</u>
C	C	A	D	B	D	C	A	A	A

Problem # 2: What are the differences between arrays and linked lists in terms of size, memory allocation time, data access, insertion/deletion of element, and memory efficiency/utilization? [0.5 x 12pts]

	Arrays	Linked lists
Size	Specified during declaration/Fixed/Static	NO need to specify; grow and shrink during execution/Dynamic
Memory allocation time	Element location is allocated during compile time	Element location is assigned during run time
Data access	Direct or randomly accessed	Sequentially accessed
Insertion/deletion of element	Slow relatively as shifting is required	Easier, Fast and efficient
Memory efficiency/Utilization	Ineffective	Efficient
Memory space	Memory required is less	Memory required is more

Problem # 3: Write a C++ program to perform deletion to the node with the maximum number in a simple unordered linked list. [5pts]

```
template <class T>
T unorderedlinklist<T>::delmax()
{
    int max_position=0;
    int counter=0;
    int max_data=0;

    node <T> *q, *r;

    q=head;

    if(head==NULL)
    {
        Cout<<"Empty List"<<endl;
        return 0;
    }

    if(q->link==NULL)
    {
        head=NULL;
        delete q;
        return 0;
    }

    max_data=q->data;
    while(q!=NULL)
    {
        if(q->data > max_data)
        {
            max_position=counter;
            max_data=q->data;
        }
        ++counter;
        q=q->link;
    }

    del(max_data);
    return 0;
}
```

Problem # 4: If the C++ program below represents the implementation of the ordered linked list, write the output of this code [0.5 x 18pts]

```
#include <iostream>
using namespace std;
template <class T>
class orderedlinklist
{
private:
template <class U>
struct node
{
T data;
node <U> *link;
};
node <T> *head;
public:
orderedlinklist();
void insert( T num );
void modifybeforeinsert(T num1, T num2);
T del( T num );
void display();
T count();
~orderedlinklist();
};
template <class T>
orderedlinklist<T>::orderedlinklist()
{
head = NULL;
}
template <class T>
void orderedlinklist<T>::insert(T num)
{
node <T>*q,*t,*n;
n= new node <T>;
n->data=num;
if( head == NULL )
{
head=n;
head->link = NULL;
}
else if(num<head->data)
{
n->link=head;
head=n;
}
else
{
q=head;
t=head->link;
while(t!=NULL && num>t->data)
```

```

{
q=t;
t=t->link;
}
q->link=n;
n->link=t;
}
}
template <class T>
void orderedlinklist<T>::modifybeforeinsert(T num1, T num2)
{
node <T>*q,*t,*n;
n= new node <T>;
if(num2 >= 0)
    n->data=num1 - num2;
else
    n->data=num1 + num2;
num1 = n->data;
if( head == NULL )
{
head=n;
head->link = NULL;
}
else if(num1<head->data)
{
n->link=head;
head=n;
}
else
{
q=head;
t=head->link;
while(t!=NULL && num1>t->data)
{
q=t;
t=t->link;
}
q->link=n;
n->link=t;
}
}
template <class T>
T orderedlinklist<T>::del( T num )
{
node <T>*q,*r;
q = head;
if( q->data == num )
{
head = q->link;
delete q;
return 0;
}
r = q;
while( q!=NULL )

```

```
{
if( q->data == num )
{
r->link = q->link;
delete q;
return 0;
}
r = q;
q = q->link;
}
cout<<"Element "<<num<<" not Found.";
return 1;
}
template <class T>
void orderedlinklist<T>::display()
{
node <T>*q;
cout<<endl;
for( q = head ; q != NULL ; q = q->link )
cout<<q->data<<endl;
}
template <class T>
T orderedlinklist<T>::count() {
node <T>*q;
int c=0;
for( q=head ; q != NULL ; q = q->link )
c++;
return c;
}
template <class T>
orderedlinklist<T>::~~orderedlinklist() {
node <T>*q;
if( head == NULL )
return;
while( head != NULL )
{
q = head->link;
delete head;
head = q;
}
}

int main()
{
orderedlinklist <int>ll;

ll.insert(48);
ll.insert(30);
ll.insert(90);
ll.modifybeforeinsert(20, 1);
ll.modifybeforeinsert(32, -1);
ll.modifybeforeinsert(92, 1);
ll.modifybeforeinsert(48, -2);
```

```
ll.insert(51);
ll.insert(19);
ll.insert(20);
ll.del(20);
ll.modifybeforeinsert(17, -2);
ll.del(46);
ll.del(90);
ll.modifybeforeinsert(20, 4);
ll.modifybeforeinsert(15, -4);
ll.modifybeforeinsert(19, -4);
ll.insert(22);
ll.modifybeforeinsert(15, -3);
ll.insert(20);
ll.insert(21);
ll.modifybeforeinsert(20, -4);
ll.del(48);
ll.del(35);
ll.modifybeforeinsert(24, 4);
ll.display();
ll.del(23);
return 0;
}
```

Output

```
Element 35 not Found.
11
12
15
15
16
16
19
19
20
20
21
22
30
31
51
91
Element 23 not Found.
```

Good Luck